Lab Worksheet 4

CSE461: Introduction to Robotics

Department of Computer Science and Engineering

Lab 04: Interfacing Arduino and Raspberry PI using UART protocol to control an LED with an LDR sensor.

I. Topic Overview

This lab introduces the fundamentals of cross-platform microcontroller interfacing using the Arduino Uno and Raspberry PI. Students will learn to establish bidirectional serial communication (UART) between the two devices, enabling them to integrate Arduino's real-time I/O control with the Raspberry PI's computational capabilities. Through hands-on exercises, students will explore voltage-level shifting, GPIO pin functionalities, and protocol-based data exchange. By the end of the lab, students will design a system where a Raspberry PI sends commands to an Arduino to control output devices based on input data, fostering a practical understanding of hybrid embedded system design.

II. Learning Outcome

After this lab, students will be able to:

1. Interface Arduino and Raspberry PI using UART protocol.
2. Design circuits that address voltage compatibility between 3.3V and 5V systems.
3. Develop code for cross-platform communication to solve real-world embedded system challenges.
4. Critically troubleshoot hardware and software integration issues.

III. Materials

- Arduino Uno R3
- Raspberry PI 4
- LED (Light Emitting Diode)
- LDR sensor
- Resistor (220Ω for LED)
- Breadboard
- Jumper wires

IV. UART Protocol Overview

UART (Universal Asynchronous Receiver-Transmitter) is a serial communication protocol that enables data exchange between microcontrollers without needing a clock signal. It uses two main lines for communication: TX (Transmit) and RX (Receive). Data is sent **byte-by-byte** in an asynchronous manner, making it a simple and efficient method for microcontroller communication. UART Pins on Arduino and Raspberry PI are as follows:

- **Arduino Uno:**
  - TX (Transmit) → Pin 1
  - RX (Receive) → Pin 0
- **Raspberry PI 4 (GPIO Header):**
  - TX (Transmit) → GPIO14 (Pin 8)
  - RX (Receive) → GPIO15 (Pin 10)

For successful communication, the **TX pin of one device must be connected to the RX pin of the other**, and vice versa.

V. Why is this lab important?

In modern embedded systems, multiple microcontrollers often work together to achieve complex tasks. This lab provides hands-on experience in cross-platform communication, an essential skill in robotics and IoT applications. By interfacing an Arduino Uno (optimized for real-time control) with a Raspberry PI (optimized for computation and networking), we will learn how to leverage

the strengths of both platforms. Understanding UART communication enables us to design hybrid embedded systems where microcontrollers exchange data efficiently, paving the way for advanced robotics, automation, and IoT projects.

VI. Setting Up the Arduino IDE in Raspberry PI

Before starting the experiments, ensure the Arduino IDE is properly set up on your Raspberry PI. To do so, refer to the link given: Setting up Arduino IDE in Raspberry PI

VII. Enabling UART on Raspberry PI

- Open the terminal on the Raspberry PI.
- Run the following command to enable UART:

```
sudo raspi-config
```

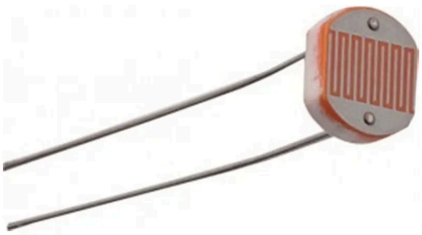
- Navigate to **Interfacing Options → Serial Port.**
- **Disable** the **login shell** over serial - Select No [Important].
- **Enable** the **serial port hardware** - Select Yes
- Reboot the Raspberry PI:

```
sudo reboot
```

VIII. Experiment 1: Using an LDR (Light Dependent Resistor) to Measure Light Intensity

1. **Description the components:**
   - Resistors (220Ω)
   - LDR

A **Light Dependent Resistor (LDR)**, also known as a **photoresistor**, is a type of resistor whose resistance changes with the intensity of light falling on it. When the surrounding light intensity increases, the resistance of the LDR decreases, allowing more current to pass through. Conversely, when the light intensity decreases, its resistance increases, reducing current flow.

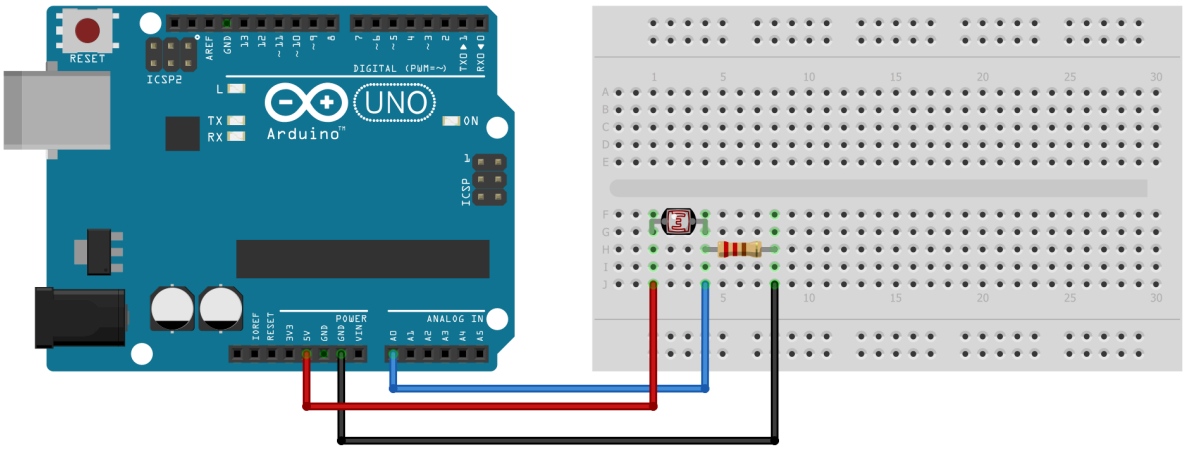


This property allows the LDR to be used in **light-sensitive applications** such as automatic streetlights, light meters, and security systems.

➢ **Bright Light** → Low resistance

➢ **Dark/ Dim Light** → High Resistance

2. **Setting up the circuit:**
   - Connect one leg of the LDR to **5V** on the Arduino and the other leg to **Analog Pin A0**.
   - Connect one leg of the **220 Ω resistor** to **A0** and the other leg to **GND**.
3. **Circuit diagram:**



4. **Code:**

```
int ldrPin = A0;
int ldrValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  ldrValue = analogRead(ldrPin); // Read analog value (0 - 1023)
  Serial.print("LDR Value: ");
  Serial.println(ldrValue); // Print value to Serial Monitor
  delay(500);
}
```

How this works: *The Arduino UNO has a 10-bit ADC (Analog-to-Digital Converter) that converts the analog voltage at pin A0 into a digital value between 0 and 1023. 0 means 0V and 1023 means 5V. The LDR and the resistor form a voltage divider. The voltage at A0 depends on the LDR's resistance, which changes with light intensity. The analogRead() value is sent to the Serial Monitor using the Serial.print() commands, allowing real-time observation of how light intensity affects the voltage across the LDR.*

IX. Experiment 2: Controlling an LED in Raspberry PI using an LDR in Arduino

5. **Description the components:**
   - LED
   - LDR
   - Resistors (220Ω for LED and Voltage Divider)

6. **Setting up the circuit:**

6.1. Arduino Uno Setup:

- Connect one leg of the LDR to **5V** on the Arduino and the other leg to **Analog Pin A0**.
- Connect one leg of the **220 Ω resistor** to **A0** and the other leg to **GND**.

6.2. Raspberry PI 4 Setup:

- Connect the LED anode (long leg) to a 220Ω resistor and the other end of the resistor to GPIO 18 (PWM Pin) on the Raspberry PI.
- Connect the LED cathode (short leg) to a GND pin.

[IMPORTANT] Connect the UART Pins (mentioned in Step 2.3) **ONLY** after uploading the code to Arduino.

Why is this step important: *Arduino's TX/RX pins (0 and 1) are used for both USB uploads and serial communication. If the Raspberry PI is already connected to these pins, it can block the Arduino from receiving new code, causing upload failures. By uploading the code first and then connecting the UART pins, we avoid this conflict and ensure both uploading and serial communication work properly*

6.3. UART Connection Between Raspberry PI and Arduino:

- Connect the TX (Transmit) pin of the Raspberry PI (GPIO 14, pin 8) to the RX (Receive) pin of the Arduino.

- Connect the RX (Receive) pin of the Raspberry PI (GPIO 15, pin 10) to the TX (Transmit) pin of the Arduino through a voltage divider.
- **[IMPORTANT]** Connect the GND pins of both devices together **(Common GND)**

Note: The Raspberry PI uses 3.3V logic, while the Arduino uses 5V logic. To avoid damaging the Raspberry PI, we are using a voltage divider to step down the 5V signal from the Arduino to 3.3V for the Raspberry PI.

7. **Circuit diagram:**

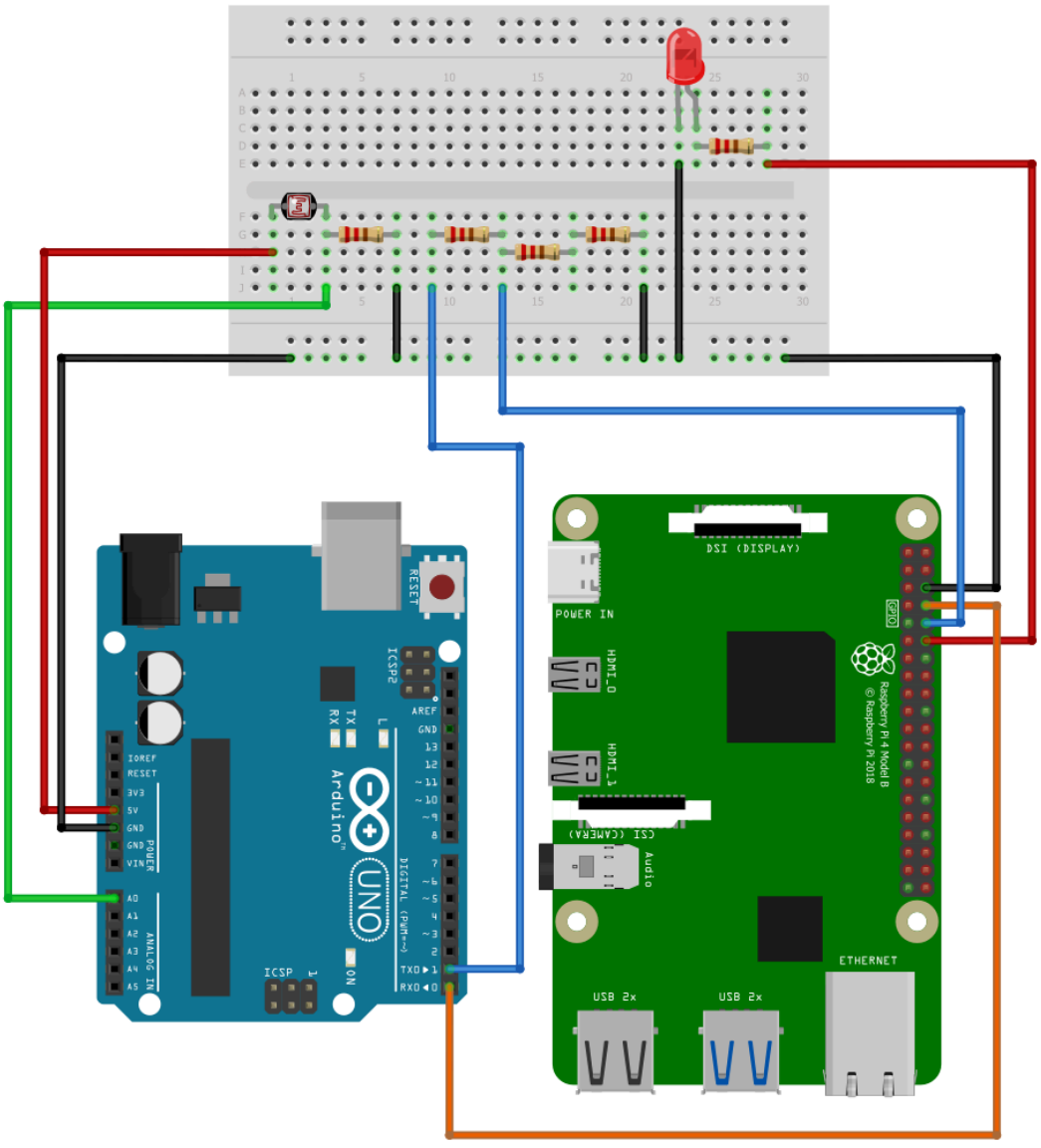


Fig. Arduino and Raspberry PI interfacing

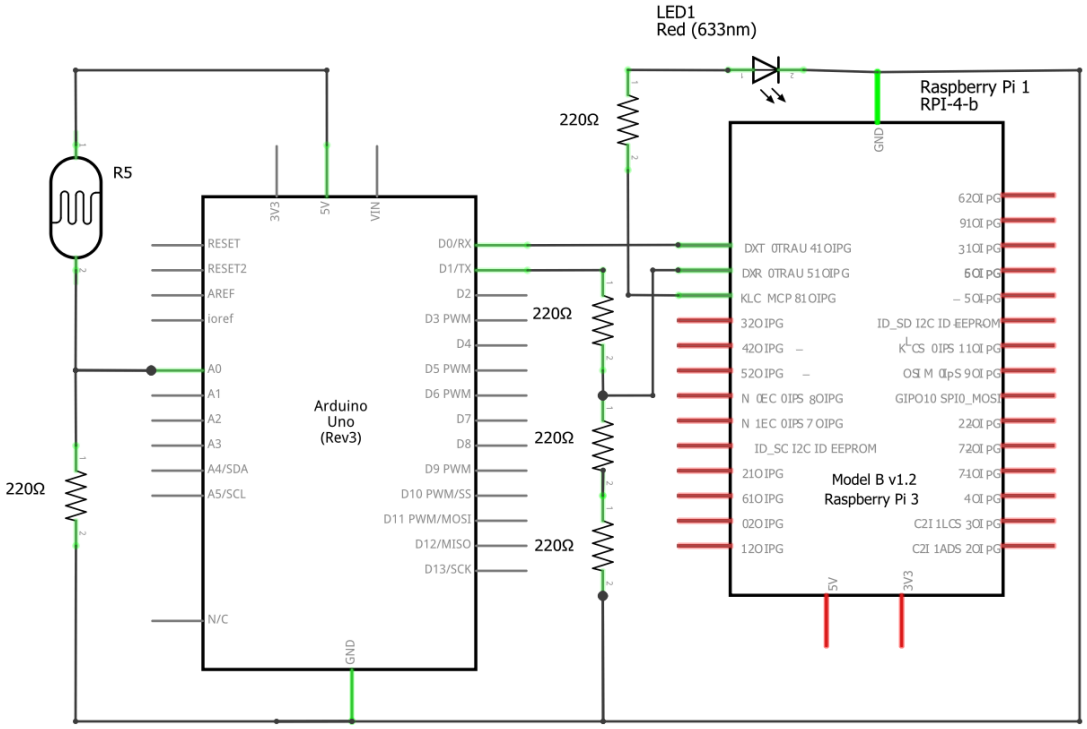
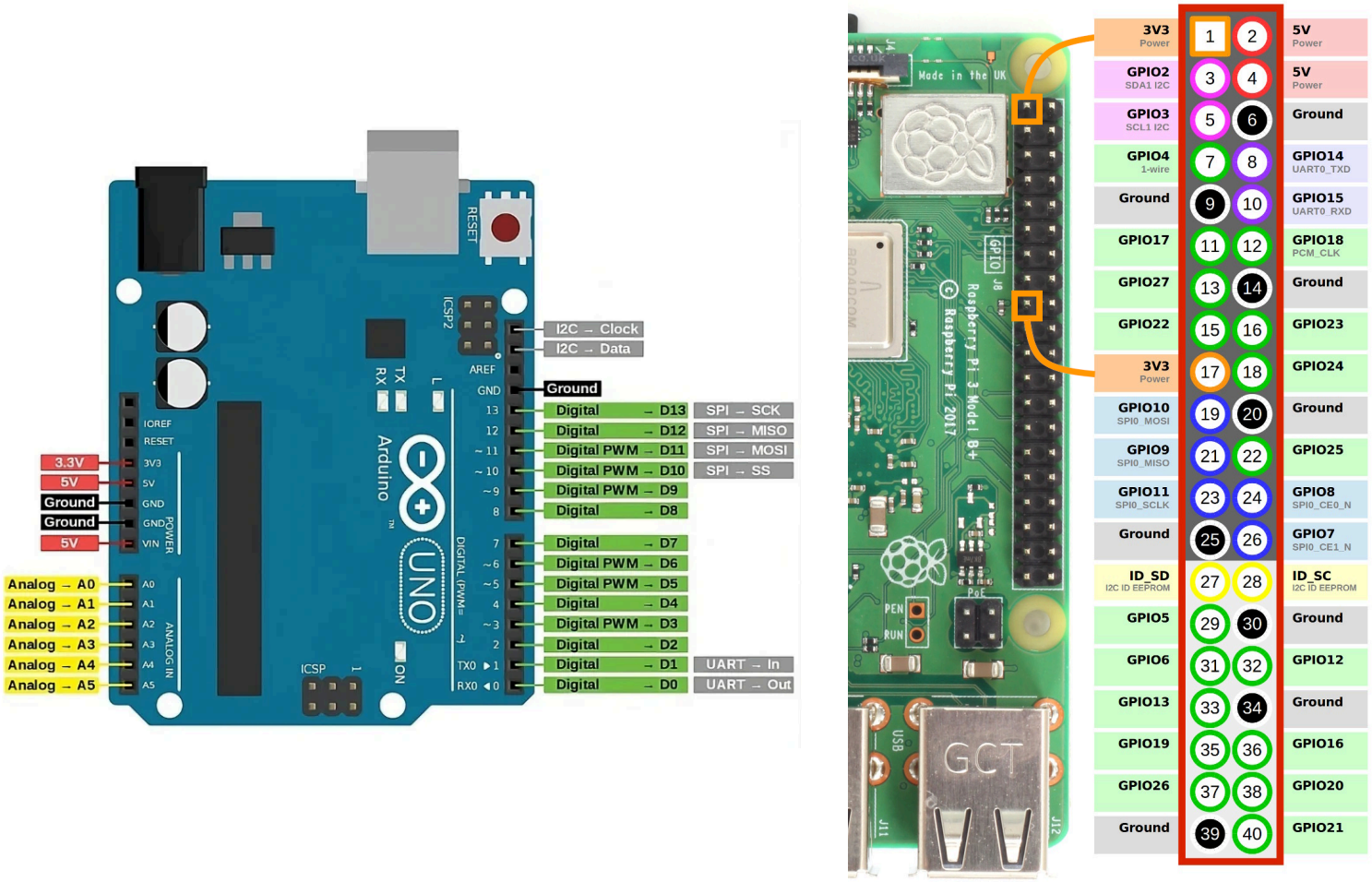


Fig. Arduino and Raspberry PI interfacing Schematic Diagram



8. **Code:**

Arduino Code:

```
int ldrPin = A0;
int ldrValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  ldrValue = analogRead(ldrPin); // Read LDR value (0-1023)
  Serial.println(ldrValue); // Send value to Raspberry Pi
  delay(100);
}
```

How this works: *Arduino reads the analog voltage from the LDR connected to A0. As light intensity changes, the LDR's resistance changes which alters the voltage at A0. The analogRead() function converts this voltage (0–5V) into a digital value between 0 and 1023 using the Arduino's 10-bit ADC. The code then sends this value to the Raspberry Pi through serial communication at 9600 baud, allowing the Pi to interpret the brightness level of the environment.*

Raspberry PI Code:

```
import serial
import RPi.GPIO as GPIO
import time

# Setup
GPIO.setmode(GPIO.BCM)
led_pin = 18
GPIO.setup(led_pin, GPIO.OUT)

# Initialize PWM
pwm_led = GPIO.PWM(led_pin, 1000)  # 1 kHz frequency
pwm_led.start(0)

# Connect to Arduino
arduino = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
time.sleep(2)
norm = 2.5

try:
    while True:
        data = arduino.readline().decode('utf-8').strip()
        if data.isdigit():
            ldr_value = int(data)
            brightness = ((1-(ldr_value / 1023)) ** norm) * 100
            Brightness = max(0, min(brightness, 100))
```

```
            pwm_led.ChangeDutyCycle(brightness)
            print(f"LDR: {ldr_value}, LED Brightness: {brightness:.1f}%")
except KeyboardInterrupt:
    print("Exiting...")
finally:
    pwm_led.stop()
    GPIO.cleanup()
    arduino.close()
```

How this works: *Raspberry Pi reads light intensity data from the Arduino via serial communication and adjusts the LED brightness connected to GPIO pin 18 using PWM. The code inverts the reading of LDR so that in darkness (low LDR value), the LED becomes brighter, and in bright light, it dims. The norm variable (2.5) applies a nonlinear scaling to make brightness changes more visible by emphasizing darker-to-brighter transitions. Finally, the calculated brightness value is limited between 0 and 100, converted to a PWM duty cycle, and applied to the LED, producing a smooth, perceivable change in brightness according to ambient light levels.*

X. Lab Task: Complete the experiment shown in class properly. That will serve as the lab task for this week.

Deliverables:

- Circuit diagram
- Raspberry PI and Arduino code
- Demonstration of the working circuit

XI. References:

1. Arduino Official Documentation: https://www.arduino.cc/en/Guide
2. Arduino IDE Download: https://www.arduino.cc/en/software
3. Raspberry PI Official Documentation : https://www.raspberrypi.com/documentation/computers/getting-started.html